

IF2224: Teori Bahasa Formal dan Automata
Pembuatan Syntax Analyser
Untuk Bahasa Pemrograman Arion



Dipersiapkan oleh:

13524008	Agatha Tatianingseto
13524030	Irvin Tandiarrang Sumual
13524074	Bernhard Aprillio Pramana
13524096	Moreno Syawali Ganda Sugita
13524110	Jennifer Khang

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
JL. GANESA 10, BANDUNG 40132
2025

Bab 1

Deskripsi Tugas

Melanjutkan tahapan kompilasi setelah *lexical analysis* selesai, sebuah program *compiler* akan memasuki tahap *syntax analysis* yang sering disebut sebagai *parser*. Jika *lexer* bertugas memecah teks (*source code*) menjadi sekumpulan token, maka *parser* bertugas memeriksa urutan token-token tersebut agar sesuai dengan *grammar* dari bahasa pemrograman yang dikompilasi. Tahapan ini sangat esensial untuk memastikan bahwa urutan token membentuk struktur sintaks yang valid secara gramatikal sebelum diolah lebih lanjut.

Pada tahap *syntax analysis* ini, *parser* akan membangun struktur sintaks berupa *Parse Tree* (Pohon Urai). *Parse Tree* adalah representasi hierarkis lengkap dari struktur program yang memuat semua simbol terminal dan non-terminal sesuai dengan *grammar* bahasa yang digunakan. Untuk mengenali struktur gramatikal dalam deretan token tersebut, *parser* menggunakan algoritma *Recursive Descent*. Algoritma ini bekerja secara *top-down* dengan memetakan setiap aturan produksi (*grammar*) ke dalam fungsi-fungsi yang saling memanggil secara rekursif.

Dalam konteks tugas ini, Arion Compiler - Syntax Analysis merupakan pengembangan tahap kedua dari kompilator bahasa pemrograman Arion. Program yang dibuat harus mampu menerima masukan berupa daftar token yang dihasilkan oleh *lexer* pada milestone sebelumnya. Dengan menggunakan metode *Recursive Descent*, program mampu menghasilkan keluaran berupa *Parse Tree* yang terstruktur serta mampu mendeteksi dan melaporkan kesalahan sintaks (*syntax error*) dengan informatif. Keluaran *Parse Tree* ini sangat krusial karena akan menjadi masukan dan fondasi bagi tahap selanjutnya dalam proses kompilasi, yaitu *semantic analysis*.

Bab 2

Teori Singkat

2.1. Syntax Analysis (Parser)

Syntax analysis, atau yang sering dikenal dengan istilah *parsing*, merupakan tahapan kedua dalam proses kompilasi sebuah bahasa pemrograman. Setelah tahapan awal (*lexical analysis*) memecah kode sumber program menjadi unit-unit terkecil yang disebut token, *syntax analysis* bertugas mengambil deretan token tersebut sebagai masukannya. Tujuan utama dari tahap ini adalah untuk memastikan dan memvalidasi bahwa token-token tersebut disusun secara bermakna sesuai dengan aturan tata bahasa (*grammar*) yang berlaku pada bahasa pemrograman tersebut. Tahapan ini menutupi kekurangan dari *lexical analysis* yang tidak mampu mendeteksi kesalahan struktur yang kompleks, seperti keseimbangan tanda kurung atau blok kode yang bersarang (*nested structures*). Jika terdapat struktur kode yang menyalahi aturan (misalnya kurangnya titik koma), *parser* akan menangkapnya sebagai *syntax error* dan melaporkannya.

2.2. Context-Free Grammar

Spesifikasi sintaks atau tata bahasa untuk sebuah bahasa pemrograman didefinisikan secara formal menggunakan *Context-Free Grammar* (CFG). CFG pada dasarnya terdiri dari kumpulan simbol terminal (token dasar seperti ifsy atau intcon), simbol non-terminal (variabel sintaks yang merepresentasikan hierarki kode seperti STATEMENT atau EXPRESSION), simbol awal (*start symbol*), dan aturan produksi (*production rules*). Aturan produksi inilah yang menjadi pedoman bagi *parser* untuk menentukan bagaimana sebuah simbol non-terminal dapat diturunkan atau diganti menjadi kombinasi deretan terminal maupun non-terminal lainnya.

2.3. Parse Tree

Apabila struktur kode dari deretan token dinyatakan sah secara gramatikal berdasarkan aturan CFG, *parser* akan merepresentasikan hierarki penurunannya ke dalam bentuk struktur pohon yang disebut *Parse Tree*. *Parse Tree* menunjukkan secara visual mengenai langkah demi langkah bagaimana sebuah kode dibentuk mulai dari simbol awal tata bahasanya. Pada struktur diagram pohon ini, bagian akar selalu melambangkan

simbol awal dari program, simpul-simpul di bagian dalam (*internal nodes*) merepresentasikan simbol *non-terminal*, dan bagian daun (*leaf nodes*) melambangkan simbol *terminal* (token aktual yang terbaca).

2.4. Recursive Descent Parsing

Secara umum, *syntax analysis* dapat dikerjakan menggunakan dua teknik utama, yaitu *Top-Down Parsing* (membangun pohon mulai dari akar lalu turun menuju daun) dan *Bottom-Up Parsing* (membangun pohon mulai dari daun lalu naik ke arah akar). *Recursive descent parsing* adalah salah satu metode implementasi dari pendekatan *Top-Down Parsing*. Algoritma ini bekerja dengan cara menerjemahkan setiap aturan produksi *non-terminal* ke dalam sebuah fungsi atau prosedur pemrograman secara langsung. Fungsi-fungsi inilah yang kemudian akan dipanggil secara berulang (rekursif) dari tingkat teratas untuk memproses dan mencocokkan deretan token yang di-supply oleh tahap lexer.

2.5. Panic Mode Recovery

Panic Mode Recovery merupakan strategi *error recovery* yang diterapkan pada *parser* untuk menangani kondisi di mana token yang terbaca tidak sesuai dengan aturan produksi yang diharapkan. Mekanisme ini bekerja dengan cara menginstruksikan *parser* untuk mengabaikan (*skip*) deretan token masukan hingga ditemukan sebuah "token sinkronisasi" yang menandakan titik aman untuk melanjutkan proses *parsing*.

Tujuan utama dari penerapan metode ini adalah untuk mencegah berhentinya program secara paksa saat menemukan kesalahan sintaks pertama. Pada kompilator Arion ini, titik sinkronisasi difokuskan pada token *FOLLOW Set*, yaitu simbol-simbol yang menandakan akhir dari sebuah *statement* atau blok tata bahasa. Token sinkronisasi yang digunakan meliputi semicolon, endsy, elsesy, dan untilsy. Ketika mendeteksi kesalahan, *parser* akan membuang token yang tidak valid hingga menemukan salah satu dari token penutup tersebut untuk kemudian melanjutkan proses kompilasinya.

Bab 3

Aturan Tata Bahasa (Grammar)

3.1. Grammar bahasa Arion

Catatan: teks yang dicetak tebal merupakan sebuah terminal, sementara yang tidak merupakan sebuah non-terminal.

Simbol Non-Terminal	Deskripsi	Aturan Produksi
<program>	Node root yang merepresentasikan keseluruhan program Arion	program $\rightarrow$ program-header + declaration-part + compound-statement + period
<program-header>	Bagian kepala program yang berisi nama program	programsy + ident + semicolon
<declaration-part>	Bagian deklarasi yang dapat berisi const, type, var, procedure, function	(const-declaration)* + (type-declaration)* + (var-declaration)* + (subprogram-declaration)*
<const-declaration>	Deklarasi konstanta	constsy + (ident + eq + constant + semicolon)+
<constant>	Konstanta dapat berupa ident , intcon , realcon , charcon , atau string	charcon string [(plus minus)? + (ident intcon realcon)]
<type-declaration>	Deklarasi tipe data baru	typesy + (ident + eq + type + semicolon)+
<var-declaration>	Deklarasi variabel	varsy + (identifier-list + colon + type + semicolon)+
<identifier-list>	Daftar identifier yang dipisahkan koma	ident (comma + ident)*
<type>	Tipe data yang terdefinisi atau didefinisikan pengguna. Dapat berupa ident , array-type, range, atau enumerated	ident array-type range enumerated record-type
<array-type>	Definisi tipe array	arraysy + lbrack + (range ident) + rbrack + ofsy + type

<range>	Rentang nilai untuk array atau subrange	constant + period + period + constant
<enumerated>	Nilai tertentu yang ada di dalam list	lparent + ident + (comma + ident)* + rparent
<record-type>	Tipe data template, sama dengan <i>class</i> dalam OOP	recordsy + field-list + endsy
<field-list>	Penjabaran atribut/field dari record	field-part + (semicolon + field-part)*
<field-part>	Singular atribut/field dari record	identifier-list + colon + type
<subprogram-declaration>	Deklarasi prosedur atau fungsi	procedure-declaration function-declaration
<procedure-declaration>	Deklarasi prosedur	proceduresy + ident + (formal-parameter-list)? + semicolon + block + semicolon
<function-declaration>	Deklarasi fungsi	functionsy + ident + (formal-parameter-list)? + colon + ident + semicolon + block + semicolon
block	Bagian kode yang mengandung deklarasi dan statement	declaration-part + compound-statement
<formal-parameter-list>	Daftar parameter formal	lparent + parameter-group + (semicolon + parameter-group)* + rparent
<parameter-group>	Daftar parameter disertai dengan tipe data parameter	identifier-list + colon + (ident array-type)
<compound-statement>	Blok statement yang diawali begin dan diakhiri end	beginsy + statement-list + endsy
<statement-list>	Daftar statement yang dipisahkan semicolon	statement (semicolon + statement)*

<statement>	Deskripsi aksi yang harus dilakukan program komputer	(assignment-statement if-statement case-statement while-statement repeat-statement for-statement procedure/function-call)?
<variable>	Variable dapat berbentuk ident, elemen array, atau field record	ident + (component-variable)*
<component-variable>	Variable dalam bentuk element array atau field record	(lbrack + index-list + rbrack) (period + ident)
<index-list>	Penunjuk index dalam array	(intcon charcon ident) + (comma + index-list)*
<assignment-statement>	Statement penugasan nilai ke variabel	variable + becomes + expression
<if-statement>	Statement kondisional	ifsy + expression + thensy + statement + (elsesy + statement)?
<case-statement>	Statement kondisional dengan banyak kondisional	casesy + expression + ofsy + case-block + endsy
<case-block>	Deklarasi case dan statement yang harus dilakukan	constant + (comma + constant)* + colon + statement + (semicolon + case-block?)*
<while-statement>	Perulangan dengan kondisi di awal	whilesy + expression + dosy + compound-statement + semicolon
<repeat-statement>	Perulangan dengan kondisi di akhir	repeatsy + statement-list + untilsy + expression
<for-statement>	Perulangan dengan counter	forsy + ident + becomes + expression + (tosy downtosy) + expression + dosy + compound-statement
<procedure/function-call>	Pemanggilan prosedur atau fungsi	ident + (lparent + parameter-list? + rparent)?
<parameter-list>	Daftar parameter aktual saat pemanggilan	expression (comma + expression)*

<expression>	Ekspresi yang menghasilkan nilai	simple-expression (relational-operator + simple-expression)?
<simple-expression>	Ekspresi tanpa operator relasional	(plus minus)? term (additive-operator + term)*
<term>	Bagian ekspresi dengan prioritas lebih tinggi	factor (multiplicative-operator + factor)*
<factor>	Unit terkecil dalam ekspresi	ident intcon realcon charcon string (lparent + expression + rparent) (notsy + factor) procedure/function-call variable
<relational-operator>	Operator perbandingan	eq neq gtr geq lss leq
<additive-operator>	Operator penjumlahan/pengurangan	plus minus orsy
<multiplicative-operator>	Operator perkalian/pembagian	times rdiv idiv imod andsy

Bab 4

Perancangan dan Implementasi

4.1. Arsitektur Program

Modul Syntax Analysis (Parser) dirancang untuk bekerja secara terintegrasi dengan modul Lexical Analysis (Lexer) yang telah dibuat pada *milestone* sebelumnya. Arsitektur utama dibungkus dalam kelas Parser yang menerima masukan berupa koleksi token (`std::vector<Token>`) yang dihasilkan oleh lexer.

Titik masuk utama (*entry point*) dari proses *parsing* adalah metode `parse()`. Metode ini menginisiasi proses *top-down parsing* dengan memanggil `parseProgram()` yang merepresentasikan *start symbol* dari tata bahasa Arion. Setelah seluruh deretan token berhasil divalidasi dan dibangun hierarkinya, parser akan mengembalikan struktur *Concrete Syntax Tree* dari *root node* `<program>`. Program juga dilengkapi dengan fungsi `writeParseTreeToFile()` untuk menampilkan keluaran hasil *parsing* ke dalam file teks (*output*).

4.2. Struktur Data Node (CSTNodes)

Untuk merepresentasikan bentuk hierarkis dari Parse Tree, kami merancang kelas CSTNodes yang fleksibel dan bersifat *type-safe*. Kelas ini menggunakan fitur `std::variant<Token, NonTerminal>` untuk menyimpan nilai dari *node*. Pendekatan ini sangat menghemat memori karena satu *node* dapat beradaptasi menjadi tiga jenis simpul, yaitu:

1. Internal Node (Simpul Dalam): Merepresentasikan simbol *non-terminal* (menyimpan nilai NonTerminal).
2. Leaf Node (Simpul Daun): Merepresentasikan simbol *terminal* (menyimpan objek Token aktual dari lexer).
3. Error Node: Secara khusus, kelas ini juga dilengkapi atribut `bool isError_` dan `std::string errorMessage_`. Fitur ini memungkinkan parser untuk menyisipkan *node* yang berisi pesan kesalahan (*error node*) langsung ke dalam cabang pohon tanpa merusak keseluruhan hierarki Parse Tree.

Setiap objek CSTNodes memiliki atribut `std::vector<CSTNodes*> children_` untuk menyimpan *pointer* dari semua node turunannya, serta dilengkapi dengan fungsi rekursif untuk mencetak pohon berindentasi.

4.3. Implementasi Recursive Descent

Algoritma *Recursive Descent* diimplementasikan dengan memetakan setiap aturan produksi *non-terminal* menjadi fungsi rekursif. Untuk menavigasi deretan token secara *top-down*, kami membangun beberapa fungsi *helper* utama:

- `peek()` dan `peekNext()`: Mengembalikan token yang sedang diamati saat ini (atau satu langkah di depan) tanpa memajukan kursor, berfungsi sebagai *lookahead*.
- `advance()`: Memajukan kursor penunjuk token ke indeks berikutnya.
- `synchronize(std::vector<TokenType> syncSet)`: Fungsi krusial yang bertugas melompati token-token masukan hingga menemukan salah satu token penanda aman (*synchronizing token*) yang ada di dalam `syncSet` (misalnya semicolon, endsy, atau untilsy).

4.4. Mekanisme Error Handling

Alih-alih menggunakan pendekatan *fail-fast* (berhenti pada *error* pertama), kami mengimplementasikan mekanisme penanganan kesalahan tingkat lanjut yaitu Panic-Mode Error Recovery.

Saat fungsi utilitas `expect(TokenType)` mendeteksi bahwa token yang dibaca tidak sesuai dengan tata bahasa, parser tidak akan langsung memicu program *crash* atau melempar *exception*. Sebaliknya, hal berikut akan terjadi:

1. Fungsi pembentuk pesan (`makeErrorMessage`) akan mencatat baris dan kolom kesalahan beserta ekspektasi tokennya.
2. Pesan ini dicatat secara global menggunakan fungsi `addError()`.
3. Parser membuat sebuah objek `errorNode` khusus untuk diletakkan ke dalam Parse Tree.
4. Parser kemudian memanggil fungsi `synchronize(...)` untuk membuang token-token tak terduga sampai ia menemukan titik sinkronisasi yang aman (contoh: titik koma ;).
5. Setelah sinkronisasi tercapai, parser akan melanjutkan proses kompilasinya.

Mekanisme ini memungkinkan kompilator Arion untuk mendeteksi, menangkap, dan melaporkan lebih dari satu kesalahan sintaks sekaligus dalam satu kali eksekusi program.

Bab 5

Pengujian

Pengujian 1

1. Nama Pengujian : spektc
2. Input file : spektc.txt
3. Output file : spektc-parse-tree.txt
4. Deskripsi : Memeriksa keakuratan program dengan contoh pada spesifikasi milestone 2
5. Hasil Pengujian :

Tabel 5.1 Pengujian 1

Input	Output
<pre> program Hello; var a, b: integer; begin a := 5; b := a + 10; writeln('Result = ', b); end. </pre>	<pre> === ARION PARSER === Membuka file: ../test/milestone-2/valid_test/spektc.txt Processing lexer... Output berhasil disimpan di: ../test/milestone-2/output/spektc-result.txt Processing parser... <program> ├─ <program-header> ├─ programsy ├─ ident(Hello) └─ semicolon ├─ <declaration-part> └─ <var-declaration> ├─ varsy └─ <identifier-list> ├─ ident(a) └─ comma </pre>

		└─	ident(b)	
		└─	colon	
		└─	<type>	
		└─	ident(integer)	
		└─	semicolon	
	└─		<compound-statement>	
		└─	beginsy	
		└─	<statement-list>	
			└─ <statement>	
			└─ <assignment-statement>	
			└─ <variable>	
				└─ ident(a)
			└─	becomes
			└─	<expression>
			└─	<simple-expression>
			└─	<term>
			└─	<factor>
			└─	intcon(5)
			└─	semicolon
			└─	<statement>
			└─	<assignment-statement>
			└─	<variable>
				└─ ident(b)
			└─	becomes
			└─	<expression>
			└─	<simple-expression>
			└─	<term>
				└─ <factor>
				└─ <variable>
				└─ ident(a)

```

| | |      |— <additive-operator>
| | |      |  |— plus
| | |      |  |— <term>
| | |      |    |— <factor>
| | |      |      |— intcon(10)
| | |— semicolon
| |— <statement>
| |  |— <procedure/function-call>
| |  |— ident(writeln)
| |  |— lparent
| |  |— <parameter-list>
| |  |  |— <expression>
| |  |  |  |— <simple-expression>
| |  |  |  |  |— <term>
| |  |  |  |  |  |— <factor>
| |  |  |  |  |    |— string(Result = )
| |  |  |  |— comma
| |  |  |  |— <expression>
| |  |  |    |— <simple-expression>
| |  |  |    |  |— <term>
| |  |  |    |  |  |— <factor>
| |  |  |    |  |  |  |— <variable>
| |  |  |    |  |  |    |— ident(b)
| |  |  |— rparent
| |  |— semicolon
| |  |— <statement>
|  |— endsy
|— period

```

Parse tree berhasil disimpan di:

../test/milestone-2/output/spektc-parse-tree.txt

	=== Parser Errors === Selesai.
--	--

Pengujian 2

1. Nama Pengujian : ifelse
2. Input file : ifelse.txt
3. Output file : ifelse-parse-tree.txt
4. Deskripsi : Pengujian valid terhadap penggunaan if else statement
5. Hasil Pengujian :

Tabel 5.2 Pengujian 2

Input	Output
<pre> program ifelse; var age: integer; begin age := 25; if age >= 18 then writeln('Adult') else writeln('Minor'); writeln('Done') end. </pre>	<pre> === ARION PARSER === Membuka file: ../test/milestone-2/valid_test/ifelse.txt Processing lexer... Output berhasil disimpan di: ../test/milestone-2/output/ifelse-result.txt Processing parser... <program> ├─ <program-header> ├─ programsy └─ ident(ifelse) └─ semicolon ├─ <declaration-part> └─ <var-declaration> ├─ varsy └─ <identifier-list> └─ ident(age) └─ colon └─ <type> └─ ident(integer) └─ semicolon └─ <compound-statement> └─ beginsy </pre>

					└─ <statement-list>
					└─ <statement>
					└─ <assignment-statement>
					└─ <variable>
					└─ ident(age)
					└─ becomes
					└─ <expression>
					└─ <simple-expression>
					└─ <term>
					└─ <factor>
					└─ intcon(25)
					└─ semicolon
					└─ <statement>
					└─ <if-statement>
					└─ ifsy
					└─ <expression>
					└─ <simple-expression>
					└─ <term>
					└─ <factor>
					└─ <variable>
					└─ ident(age)
					└─ <relational-operator>
					└─ geq
					└─ <simple-expression>
					└─ <term>
					└─ <factor>
					└─ intcon(18)
					└─ thensy
					└─ <statement>
					└─ <procedure/function-call>

						└─ ident(writeln)
						└─ lparent
						└─ <parameter-list>
						└─ <expression>
						└─ <simple-expression>
						└─ <term>
						└─ <factor>
						└─ string(Adult)
						└─ rparent
						└─ elsesy
						└─ <statement>
						└─ <procedure/function-call>
						└─ ident(writeln)
						└─ lparent
						└─ <parameter-list>
						└─ <expression>
						└─ <simple-expression>
						└─ <term>
						└─ <factor>
						└─ string(Minor)
						└─ rparent
						└─ semicolon
						└─ <statement>
						└─ <procedure/function-call>
						└─ ident(writeln)
						└─ lparent
						└─ <parameter-list>
						└─ <expression>
						└─ <simple-expression>
						└─ <term>

```
| | | | <factor>
| | | | string(Done)
| | | | rparent
| | | | endsy
| | | | period
```

Parse tree berhasil disimpan di:

../test/milestone-2/output/ifelse-parse-tree.txt

=== Parser Errors ===

Selesai.

Pengujian 3

1. Nama Pengujian : subprogram
2. Input file : subprogram.txt
3. Output file : subprogram-parse-tree.txt
4. Deskripsi : Menguji kondisi valid untuk subprogram (procedure/function)
5. Hasil Pengujian :

Tabel 5.3 Pengujian 3

Input	Output
program subprogram; procedure p(x: integer); begin end; function f: real; begin p(10); end.	=== ARION PARSER === Membuka file: ../test/milestone-2/valid_test/subprogram.txt Processing lexer... Output berhasil disimpan di: ../test/milestone-2/output/subprogram-result.txt Processing parser... <program> ├─ <program-header> └─ programsy ├─ ident(subprogram) └─ semicolon ├─ <declaration-part> └─ <subprogram-declaration> └─ <procedure-declaration> ├─ proceduresy ├─ ident(p) ├─ <formal-parameter-list> ├─ lparent ├─ <parameter-group> ├─ <identifier-list> └─ ident(x)

						└─ colon
						└─ ident(integer)
						└─ rparent
						└─ semicolon
						└─ <block>
						└─ <declaration-part>
						└─ <compound-statement>
						└─ beginsy
						└─ <statement-list>
						└─ <statement>
						└─ endsy
						└─ semicolon
						└─ <subprogram-declaration>
						└─ <function-declaration>
						└─ functionsy
						└─ ident(f)
						└─ colon
						└─ ident(real)
						└─ semicolon
						└─ <block>
						└─ <declaration-part>
						└─ <compound-statement>
						└─ beginsy
						└─ <statement-list>
						└─ <statement>
						└─ endsy
						└─ semicolon
						└─ <compound-statement>
						└─ beginsy
						└─ <statement-list>

```

| | | └─ <statement>
| | |   └─ <procedure/function-call>
| | |     └─ ident(p)
| | |       └─ lparent
| | |         └─ <parameter-list>
| | |           └─ <expression>
| | |             └─ <simple-expression>
| | |               └─ <term>
| | |                 └─ <factor>
| | |                   └─ intcon(10)
| | |                     └─ rparent
| | └─ semicolon
| └─ <statement>
└─ endsy
    └─ period

```

Parse tree berhasil disimpan di:

../test/milestone-2/output/subprogram-parse-tree.txt

=== Parser Errors ===

Selesai.

Pengujian 4

1. Nama Pengujian : missingsemicolon
2. Input file : semicolon.txt
3. Output file : semicolon-parse-tree.txt
4. Deskripsi : Pengujian error jika terdapat semicolon yang hilang
5. Hasil Pengujian :

Tabel 5.4 Pengujian 4

Input	Output
<pre> program missingsemicolon; var a, b: integer; begin a := 5 b := 10; writeln(a, b) end. </pre>	<pre> === ARION PARSER === Membuka file: ../test/milestone-2/invalid_test/semicolon.txt Processing lexer... Output berhasil disimpan di: ../test/milestone-2/output/semicolon-result.txt Processing parser... <program> ├── <program-header> ├── programsy ├── ident(missingsemicolon) └── semicolon ├── <declaration-part> └── <var-declaration> ├── varsy ├── <identifier-list> ├── ident(a) ├── comma └── ident(b) ├── colon ├── <type> └── ident(integer) </pre>

```

|   └─ semicolon
└─ <compound-statement>
|   └─ beginsy
|   └─ <statement-list>
|       └─ <statement>
|           └─ <assignment-statement>
|               └─ <variable>
|                   └─ ident(a)
|                       └─ becomes
|                           └─ <expression>
|                               └─ <simple-expression>
|                                   └─ <term>
|                                       └─ <factor>
|                                           └─ intcon(5)
└─ <unknown:Syntax error at line 5, col 3: expected
'endsy', got 'ident'>
    └─ <unknown:Syntax error at line 6, col 3: expected 'period',
got 'ident'>

Parse tree berhasil disimpan di:
../test/milestone-2/output/semicolon-parse-tree.txt

=== Parser Errors ===
Syntax error at line 5, col 3: expected 'endsy', got 'ident'
Syntax error at line 6, col 3: expected 'period', got 'ident'

Selesai.

```


Pengujian 5

1. Nama Pengujian : toomuch
2. Input file : toomuch.txt
3. Output file : toomuch-parse-tree.txt
4. Deskripsi : Pengujian kasus invalid jika terdapat statement berlebih
5. Hasil Pengujian :

Tabel 5.5 Pengujian 5

Input	Output
<pre> program Hello; var a, b: integer else; begin a := 5;;; b := a + 10; writeln('Result = ', b); end. </pre>	<pre> === ARION PARSER === Membuka file: ../test/milestone-2/invalid_test/toomuch.txt Processing lexer... Output berhasil disimpan di: ../test/milestone-2/output/toomuch-result.txt Processing parser... <program> ├─ <program-header> ├─ programsy │ ├─ ident(Hello) │ └─ semicolon ├─ <declaration-part> └─ <var-declaration> ├─ varsy │ ├─ <identifier-list> │ │ ├─ ident(a) │ │ └─ comma │ │ └─ ident(b) │ └─ colon └─ <type> └─ ident(integer) </pre>

```
|   └─ <unknown:Syntax error at line 4, col 17: expected
'semicolon', got 'elsesy'>
| └─ <compound-statement>
|   └─ <unknown:Syntax error at line 4, col 17: expected 'beginsy',
got 'elsesy'>
|   └─ <statement-list>
|   └─ └─ <statement>
|   └─ └─ <unknown:Syntax error at line 4, col 17: expected 'endsy',
got 'elsesy'>
|   └─ └─ <unknown:Syntax error at line 4, col 17: expected 'period', got
'elsesy'>
```

Parse tree berhasil disimpan di:

../test/milestone-2/output/toomuch-parse-tree.txt

=== Parser Errors ===

Syntax error at line 4, col 17: expected 'semicolon', got 'elsesy'

Syntax error at line 4, col 17: expected 'beginsy', got 'elsesy'

Syntax error at line 4, col 17: expected 'endsy', got 'elsesy'

Syntax error at line 4, col 17: expected 'period', got 'elsesy'

Selesai.

Bab 6

Kesimpulan dan Saran

6.1. Kesimpulan

Berdasarkan hasil perancangan dan implementasi yang telah dilakukan, dapat disimpulkan bahwa:

- Telah berhasil dibuat program dalam bahasa C++ yang mengimplementasikan tahap Syntax Analysis (Parser) menggunakan metode *Recursive Descent* untuk *compiler* bahasa pemrograman Arion.
- Program ini telah memenuhi spesifikasi yang diberikan, yaitu mampu menerima masukan berupa daftar token dari *lexer* dan menghasilkan *output* berupa *Parse Tree* yang valid secara visual.
- Program dapat mendeteksi *syntax error* berdasarkan *Context-Free Grammar* Arion yang diberikan, serta menampilkan pesan kesalahan yang menunjukkan baris dan kolom letak ketidaksesuaian token tersebut secara informatif.

6.2. Saran

Beberapa hal yang dapat dikembangkan lebih lanjut untuk meningkatkan kualitas program pada pengembangan selanjutnya adalah:

- Penyederhanaan struktur pohon urai (*Concrete Syntax Tree*) menjadi *Abstract Syntax Tree* (AST) pada tahapan selanjutnya untuk membuang token yang tidak memiliki nilai semantik agar proses analisis menjadi lebih efisien.
- Penggunaan manajemen memori modern pada C++ (*smart pointers* seperti `std::unique_ptr`) untuk menyimpan dan mengelola alokasi node pohon secara dinamis agar terhindar dari potensi *memory leak*.
- Meningkatkan akurasi sinkronisasi ketika melakukan error panic mode recovery.

Lampiran

Tautan Repositori Github:

<https://github.com/Irvin-Tandiarrang-Sumual/STI-Tubes-IF2224-2026.git>

NIM	Nama	Deskripsi Tugas	Persentase Kontribusi
13524008	Agatha Tatianingseto	Kerja	20%
13524030	Irvin Tandiarrang Sumual	Kerja	20%
13524074	Bernhard Aprillio Pramana	Kerja	20%
13524096	Moreno Syawali Ganda Sugita	Kerja	20%
13524110	Jennifer Khang	Kerja	20%

Daftar Referensi

GeeksforGeeks. (2024). *Introduction to Syntax Analysis in Compiler Design*. [Online]. Available: <https://www.geeksforgeeks.org/compiler-design/introduction-to-syntax-analysis-in-compiler-design/>. Accessed 05 May 2026.

TutorialsPoint. (n.d.). *Compiler Design - Syntax Analysis*. [Online]. Available: https://www.tutorialspoint.com/compiler_design/compiler_design_syntax_analysis.htm. Accessed 05 May 2026.

GeeksforGeeks. (2024). *Recursive Descent Parser*. [Online]. Available: <https://www.geeksforgeeks.org/compiler-design/recursive-descent-parser/>. Accessed 05 May 2026.

GeeksforGeeks. (2025). *Error Recovery Strategies in Compiler Design*. [Online]. Available: <https://www.geeksforgeeks.org/compiler-design/error-recovery-strategies-in-compiler-design/>. Accessed 07 May 2026.

TutorialsPoint. (n.d.). *Compiler Design - Error Recovery*. [Online]. Available: https://www.tutorialspoint.com/compiler_design/compiler_design_error_recovery.htm. Accessed 07 May 2026.